

Fundamentals of Computer Science II AE3

April 24, 2026

Introduction

This project models the journey from Gatwick airport to Heathrow Airport along the M25 motorway in a weighted graph. We then use three algorithm (BFS, DFS, and Dijkstra's) to find a route. This report compares what each one found and why.

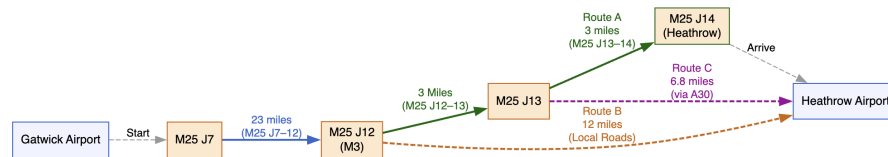


Figure 1: Visualized Graph

Best Route Between Algorithms

All three algorithms got the same amount of data but each one all returned a different "best" route. This is because each algorithm has a different idea of what the "best" route is. BFS defines best route as the fewest amount of nodes so it picked Route B, because it only had 3 nodes to get its destination. The algorithm does not care that it took 43.12 minutes to get to end, which is the slowest.

DFS does not have a definition of "best" is either. It just picks whatever path it finds first based on the order of the dictionary. It found Route A at 33.32 minutes. This happened to be faster than BFS but that was just by how I created the dictionary not due to the algorithm.

Dijkstra defines "best" as the lowest amount of travel time. It picked Route C, which came out at 30.20 minutes. This is the only algorithm that can really say it found the fastest route because it is the only one that actually uses the weights.

This just shows that the word "best" depends entirely on the situation and the problem you are trying to solve. For this project Dijkstra's was the best but if we wanted to find connections on LinkedIn for a person, BFS would be the best for that situation.

Algorithm	Path Found	Nodes	Total Time	What it Optimised For
BFS	J7 → J12 → Heathrow (Route B)	3	~43 mins	Fewest stops
DFS	J7 → J12 → J13 → J14 → Heathrow (Route A)	5	~33 mins	Nothing (first path found)
Dijkstra's	J7 → J12 → J13 → Heathrow (Route C)	4	~30 mins	Fastest travel time

Breadth First Search (BFS)

What it Optimises For

BFS guarantees that it will find the route with the fewest amount of nodes between the start and the end. It does this by using a queue (FIFO - First in First out) to go through the graph level by level. It visits every node one step away first and then every node two steps away and so on and on. Because of this, the first time BFS reaches the endpoint it is guaranteed to be the path with the fewest amount of nodes. BFS does not look at weights meaning it does not take travel time into account at all.

```
queue = deque([[first_node]])
visited = set()
visited.add(first_node)

while queue:
    path = queue.popleft()
    current = path[-1]
```

This shows BFS using `popleft()` to always process the node that was added first, showing that it explores level by level.

Route Result

BFS found Route B which is J7 -> J12 -> Heathrow, because it only cares about finding the path with the fewest amount of nodes. It goes level by level so the first time it reaches Heathrow it stops since that is the path with the fewest amount of steps to get there. It does not look at the weights at all. This is why it picked Route B because it only passes through 3 nodes making it the shortest paths in terms of steps. However the actual travel time for this route was 43.12 minutes, which was the slowest out of all three of the algorithms.

Real Word Examples

BFS is the best choice when the goal is to get somewhere in the fewest amount of steps. An example of this is social media suggestions or connections on LinkedIn. On LinkedIn, when it suggests "People you may know", it is using BFS logic. Your direct connections are one node away and then their connections are two nodes away and then so on. The algorithm does not care how close you are with someone it just cares about the number of nodes between you and a possible connection. Using an algorithm like Dijkstra's would not make sense because it's only useful when roads take different amounts of time because it always tried to find the fastest route. But if every road takes the same amount of time, there is no reason to compare every path. Each path is just the same as the others. So for this situation, Dijkstra's would just be extra work.

Depth First Search (DFS)

What it Optimises For

DFS does not optimize for anything. There is no guarantee that it will find the shortest or the fastest path. It uses simple recursion to go down one path before coming back and trying another. The path it chooses depends on the order that the neighbours are stored in the dictionary. If I were to do DFS on a different dictionary order, it could give me a completely different result. The only thing that DFS guarantees really is that it will find a path to the location if that path exists.

Route Result

DFS found Route A which is J7 -> J12 -> J13 -> J14 -> Heathrow because it does not optimise for anything. It picks the first neighbour it sees and goes as deep as possible down the route before it goes back. For this project, from J12 it visited J13 first because J13 appears before Heathrow in the dictionary. It then went to J14 and then reached Heathrow and stopped. If I had stored the neighbours in a different order, DFS could have found a completely different path. Also, if J14 was a dead end, it would have backtracked back to J12 and found another route until it reached Heathrow. The route it found took about 33.32 minutes which is faster than the BFS time but this was only really due to the way I wrote the order of the dictionary, not because DFS was trying to find the fastest route. Just like BFS, DFS does not account for weight at all.

Real Word Examples

DFS is the best for problems where the goal is to go through everything before moving on. A good example of this is trying to get through a maze. When you walk through a maze, you pick a certain direction and keep going down that path until you find the exit or you hit a dead end. If you hit the dead end, you backtrack back to the last point. You then need to find the exit again or you hit a dead end. This is how DFS works because it follows one path as far as you can go and then come back and try another path. BFS would not work here because it would try to go through every single turn which would use a lot of memory and time to get through the maze.

Dijkstra's Algorithm

What it Optimizes For

Dijkstra's algorithm guarantees that it will find the route that takes the lowest amount of time to get to the destination. It does this by using a priority queue to always go to the node that has the smallest cost. Each time it reaches a new node, it checks whether the new path then the previous path. When it reaches the destination, it checks that there is no cheaper path. The only con of Dijkstra's is that it only works when the weights are positive numbers. In this case, it works because the travel times cannot be negative.

Route Result

Dijkstra's found Route C which is J7 -> J12 -> J13 -> Heathrow with the travel time of 30.0 minutes. It picked this route because it checked every possible path and compared their total weights. Even though Route A goes through more nodes, its total time was 33.32 minutes which is higher. Route B was even slower at 43.12 minutes. Dijkstra was able to find Route C as the fastest because it is the only algorithm that actually reads and looks at the weights by using a priority queue.

Real Word Examples

Dijkstra's algorithm is the best choice when different paths have different weights and the goal is to find the cheapest option possible. The best example of this is apps like Google Maps and Waze that use this. When you ask the app for directions, the app knows that every road has different speeds like a highway is much longer but can have a higher top speed, or that an inside road would be shorter but is much slower to get through. Dijkstra's looks at all the options and then shows you the best route with the lowest travel time. This is exactly what we did for this project. BFS or DFS cannot do this because they both do not look at travel times at all.

Output in Terminal (Jan 19th, 2026)

```
Building graph from WebTRIS data...

J7 -> J12: 23.12 mins (avg speed 59.69362745098039 mph)
J12 -> J13: 2.87 mins (avg speed 62.61309523809524 mph)
J13 -> J14: 2.98 mins (avg speed 60.32118055555555 mph)
J14 -> Heathrow: 4.35 mins (avg speed 41.395833333333336 mph)
J13 -> Heathrow: 4.21 mins (avg speed 54.145833333333336 mph)
J12 -> Heathrow: 20.0 mins (hardcoded, no sensors)

Full Graph:
J7 -> {'J12': 23.12}
J12 -> {'J13': 2.87, 'Heathrow': 20.0}
J13 -> {'J14': 2.98, 'Heathrow': 4.21}
J14 -> {'Heathrow': 4.35}
Heathrow -> {}

BFS Path:
J7 -> J12 -> Heathrow ->
Number of nodes: 3
Total time: 43.12 mins

DFS Path:
J7 -> J12 -> J13 -> J14 -> Heathrow ->
Number of nodes: 5
Total time: 33.32 mins

Dijkstra's Path:
J7 -> J12 -> J13 -> Heathrow ->
Number of nodes 4
Total time: 30.2 mins
```